

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene
Faculté d'Informatique



Report

Speciality : 2nd year Computer Engineering
Module : Algorithms and Data Structures 3

Mini-project :

A game with implementation of dynamic data structures

Report Written by:

HAMDI Sarah
DJENKAL Lyna

Registration Number :

232331551005
242431414419

Supervised by:

S.Boukhedouma

Section : C - Groupe : 01
Academic year : 2025-2026

Résumé

Ce rapport présente le développement d'un jeu en C jouable dans le terminal. Le jeu utilise des structures de données pour gérer les joueurs, et comporte deux stratégies distinctes. Le rapport décrit la conception des algorithmes, leur complexité ainsi qu'une analyse des résultats, l'équité du jeu, et illustre le déroulement des parties avec des exemples issus du terminal.

- **Mots-clés :** Jeu en C, Algorithmes, Équité du Jeu, Files et Listes Chaînées, Structures de Données.

Abstract

This report presents the development of a C game playable on a terminal. The game uses data structures to manage players and features two distinct strategies. The report describes the design of the algorithms, their complexity, an analysis of the results, the game's fairness, and illustrates gameplay with examples from the terminal.

- **Keywords :** Games in C, Algorithms, Game Fairness, Queues and Linked Lists, Data Structures.

Contents

Contents	ii
List of Figures	iii
List of Algorithms	iv
Introduction	1
1 General Presentation of the Project	2
1.1 Introduction	2
1.2 Objective of the Game	2
1.3 Main Features of the Game	2
1.4 Tools and Technologies Used	3
1.5 Conclusion	3
2 Game Design	4
2.1 Introduction	4
2.2 Data and Data Structure Modeling	4
2.2.1 Player Modeling and Records	4
2.2.2 Modeling of Queues and Linked Lists	5
2.2.3 Choice of Data Structures	5
2.3 Algorithms development and Complexity	5
2.3.1 Helping functions	5
2.3.2 Algorithms of the first part	7
2.3.3 Algorithms of the second part	8
2.3.4 Combined Game functions	9
2.4 Conclusion	14
3 Implementation and realization	15
3.1 Introduction	15
3.2 Gameplay	15
3.2.1 Strategy One	15
3.2.2 Strategy Two	15
3.3 Analysis of Results	16
3.3.1 Visual excerpts from the user experience	16
3.4 Difficulties encountered and prospects for improvement	20
3.5 Conclusion	20
Conclusion	21

List of Figures

3.1	Game progress displayed from launch	17
3.2	Display of rounds and winning player	17
3.3	End of the first strategy and display of results	18
3.4	Gameplay using second strategy	18
3.5	End of the second strategy	19
3.6	End of game results	19

List of Algorithms

1	ADDtoLG	6
2	HasTwoPlayers	6
3	PlayTurn (Digit-Sum Strategy)	7
4	PlayTurn2 to start the second strategy	8
5	CreateF	9
6	GameRound	9
7	MovePlayer	9
8	Priority	10
9	Update_P_Stats	11
10	GameResults	11
11	DisplayGameState	11
12	FinishedGame	12
13	DisplayTop3Winners	12
14	DISPLAY_N_WINS	13
15	PlayGame (Main Game Loop)	14
16	PlayGame - Complexity Analysis	14

Introduction

This report presents the development of a C game displayed in the terminal. It first describes the general idea of the project and the game's main features, then details the design and implementation of the data structures used, including queues and ordered linked lists. Next, the report presents the algorithms used and an analysis of their complexity. The game contains two strategies: the first is based on the sum of digits and multiples of 5, while the second relies on calculating the greatest common divisor (GCD) and verifying digits. Finally, the report includes an analysis of the results, the game's fairness, and presents examples of gameplay as displayed in the terminal.

Chapter 1

General Presentation of the Project

1.1 Introduction

Effective data management is essential in the development of any multiplayer game, which requires the use of appropriate data structures.

In this first chapter, we will present an overview of our game, which involves multiple players participating in successive rounds. To manage player scores, progress, and rankings, we use suitable data structures, such as queues and linked lists.

1.2 Objective of the Game

The goal of our game is to manage and track players' progress based on their results across different rounds.

Each player can win or lose one or more rounds, and depending on their score, they are moved to different queues or placed on final lists.

At the end of the game, a list of winning players and a list of losing players are generated, while adhering to the defined strategy and rules regarding wins, losses, and accumulated points.

1.3 Main Features of the Game

Our game includes several key features. First, all players are placed in a main queue to participate in successive rounds. Based on their scores, some players are moved to priority queues as they record wins or losses. Players who reach a certain number of wins or losses are removed from the game and placed on final winner or loser lists. The game relies entirely on the random generation of numbers, and the strategy used determines the course of each round and impacts the scoring. Finally, the display functions allow viewing of rounds and players according to various criteria, such as: player name and age, number of wins and losses, and the start and end date and time of each game played.

1.4 Tools and Technologies Used

Our game was developed in C. The libraries used are standard C libraries, such as:

`<time.h>` : to handle the start and end date and time of each game.

`<stdbool.h>` : to work with boolean variables.

`<stdlib.h>` : to generate random values and manage memory.

These libraries make it easy to implement all the game's features.

1.5 Conclusion

This chapter laid the groundwork for our project by presenting the general concept of our game, its main features, and the tools necessary for its development. These elements form the framework upon which the detailed design and implementation of the game will be based.

Chapter 2

Game Design

2.1 Introduction

After presenting the general idea of the project and its main features, in this chapter we will explain the design of our game, the data structures used, as well as an analysis of the complexity of the algorithms we used to carry out the different parts of the project.

2.2 Data and Data Structure Modeling

This section focuses on modeling the data used in the game and choosing appropriate data structures. The goal is to represent the different game entities. The data structures were selected based on the game logic requirements, such as score management and player rankings.

2.2.1 Player Modeling and Records

The player is represented using a record (struct Player) that contains all the necessary information to follow their progress during the game and each round.

Each player has a unique number (num) which is used to identify them from other participants. Their name (name) and age (age) are also stored. Several fields are used to track player progress, such as:

- **"game_score"** and **"score"**: These store the current score in a game, and the score obtained by the player throughout the entire game.
- **"successive_point"**: This field represents the number of successive points earned by the player. This is used to move players in linked lists.
- **"nb_wins"** and **"nb_losses"**: These indicate the number of games won and lost by the player.
- **"nb_succ_wins"** and **"nb_succ_losses"**: These keep track of the number of consecutive wins or losses, which is useful for moving players in priority queues.
- **"is_winner"**: This boolean indicates whether the player is the winner of a game, and makes result displaying easier.

Other records are used to represent the nodes of linked lists and the elements of queues.

2.2.2 Modeling of Queues and Linked Lists

Queues are used to manage players according to the FIFO (First In, First Out) principle. This maintains the order in which players join the queue: the first player to enter plays first, while the last to arrive plays last. There are multiple queues in our game: the main queue F, with priority 2, contains all players participating in the game, queue F1, with priority 1, contains players who have won three consecutive games; and queue F3, with priority 3, contains players who have lost three games consecutively or separately. The fundamental operations (primitives) utilized in the queues, such as "EmptyQueue", "Display_Queue", "Init_Queue", "EnQueue", and "DeQueue", are performed using simple and efficient algorithms.

Linked lists are used to store players who have finished the game. For this, two lists are used: LP to store players who have lost five games, even if separated, and LG to store players who have won five games, even if separated. This list is ordered in descending order of the points accumulated over the five games, which allows winning players to be ranked according to their performance.

2.2.3 Choice of Data Structures

Using records allows for a clear data structure and facilitates the manipulation of information related to the different entities in the game.

The choice to use queues is based on the need to respect the order in which players arrive, first come, first served.

Linked lists, on the other hand, allow elements to be added or removed easily and dynamically at any time without resizing a fixed structure, unlike statically sized arrays.

2.3 Algorithms development and Complexity

In this section, we present the main algorithms written in an algorithmic language. These algorithms allow us to clearly describe the operation of certain key features of our game. In our project, we took care to calculate the complexity of our algorithms to get an idea of their execution speed. This allowed us to better understand the behavior of our code and to verify that it remains efficient, even when the amount of data increases.

2.3.1 Helping functions

1. **The Queue Primitives** Necessary to manipulate the three queue structures F , F1 and F3 and essential to our game functions (InitQueue , EmptyQueue , Enqueue , Dequeue , DisplayQueue). All with a constant Complexity $O(1)$.
2. **List Functions** InitList , ADDtoLP , ADDtoLG are functions aiding in initialising a list hence InitList and Adding a player to the LG (ADDtoLG) and LP (ADDtoLP) list once a player has definitely lost or won and is exempt from playing, placing the LG player depending on its succesifs_points. InitList and ADDtoLP are characterized by a constant complexity while ADDtoLP has a $O(N)$ complexity with N the number of players in LG (worst case).

Algorithm 1 ADDtoLG

```
1: Input: LG : List pointer, player : Player
2: Output: None
3: new  $\leftarrow$  AllocateMemory(sizeof(Node))
4: new.P  $\leftarrow$  player
5: if LG.L = NULL OR player.succesifs_points > LG.L.P.succesifs_points then
6:   new.Pnext  $\leftarrow$  LG.L
7:   LG.L  $\leftarrow$  new
8: else
9:   Q  $\leftarrow$  LG.L
10:  while Q.Pnext  $\neq$  NULL AND Q.Pnext.P.succesifs_points  $\geq$  player.succesifs_points
11:    do
12:      Q  $\leftarrow$  Q.Pnext
13:  end while
14:  new.Pnext  $\leftarrow$  Q.Pnext
15:  Q.Pnext  $\leftarrow$  new
16: end if
17: Complexity:  $O(n)$  where  $n$  is number of players in LG (worst case)
```

3. **The function HasTwoPlayers** aids in determining whether we can choose two players out of the same queue ; returning true if the queue has two elements , else false.

Algorithm 2 HasTwoPlayers

```
1: Input: F : Queue
2: Output: Boolean
3: if EmptyQueue(F) then
4:   return FALSE
5: end if
6: return (F.Head.Pnext  $\neq$  NULL)
```

2.3.2 Algorithms of the first part

1. **The function PlayTurn** implements the first strategy of the game by generating for a random number for the player chosen , compute the sum of its digits then seeing if it's a multiple of 5 ; if so it returns one point for the player otherwise returns 0 point.

Algorithm 3 PlayTurn (Digit-Sum Strategy)

```
1: Input : PlayerID : integer , PlayerName : String;
2: Output : point : integer;
3:  $r \leftarrow rand()1000000$ ;
4:  $sum \leftarrow 0$ ;
5:  $n \leftarrow r$ ;
6: while ( $n > 0$ ) do
     $sum \leftarrow sum + (n10)$ ;  $n \leftarrow n \div 10$ ;
7: end while;
8: if ( $sum5 = 0$ ) then
9:    $point \leftarrow 1$ ;
10: else
11:    $point \leftarrow 0$ ;
12: end if;
13:  $point$ ; Complexity:  $O(d)$  with  $d$  = number of digits
```

2.3.3 Algorithms of the second part

1. **The function PlayTurn2** implements the second strategy of the game. It generates two random numbers, computes their GCD, and checks if the GCD contains at least one digit in common with one of the two generated numbers to award a point.

Algorithm 4 PlayTurn2 to start the second strategy

```
1: Input : PlayerID : integer; PlayerName : char[20];
2: Output : point : integer; // point = 0 or 1
3: Var : a, b, n, digit, point : integer; find : boolean;
4: find  $\leftarrow$  false ;
5: a  $\leftarrow$  calling function "rand" that generates a number between 0 and 999999 ;
6: b  $\leftarrow$  calling function "rand" that generates a number between 0 and 999999 ;
7: write ("You have generated the values:");
8: n  $\leftarrow$  GCD(a, b);
9: while (n > 0) AND (find = false) do
10:   digit  $\leftarrow$  n mod 10;
11:   if (number mod 10) = digit then
12:     return true;
13:   end if
14:   nombre  $\leftarrow$  nombre / 10;
15:   // check if the digits of GCD belong to at least one of the two generated numbers
16:   if (number_contains_digit(a, digit)) OR (number_contains_digit(b, digit)) then
17:     find  $\leftarrow$  true ;
18:   end if
19:   n  $\leftarrow$  n / 10;
20: end while
21: if find = true then
22:   point  $\leftarrow$  1;
23:   write ("The player has won a point!");
24: else
25:   point  $\leftarrow$  0;
26:   write ("The player did not score any points!");
27: end if
28: return point;
```

- The initialization instructions, the generation of the two numbers, and the display of messages are simple operations. Their complexity is constant $O(1)$.
- Calculating the GCD using the recursive function has a constant complexity $O(1)$.
- The while loop iterates through the digits of the $GCD = n$. In the worst case, this loop executes a number of times equal to the number of digits in n , so the complexity is $O(n)$.
- Inside the while loop, the function `number_contains_digit` is applied to the two numbers a and b . This function browses the digits of a number with a complexity of $O(k)$, making the total complexity of the loop $O(n \times k)$.
- Finally, the complexity of this function is linear $O(n \times k)$, it depends on the number of digits in the GCD and the generated numbers a and b .

2.3.4 Combined Game functions

1. **The function CreateF** generates the initial player pool with randomized names and ages to start the tournament. The function initializes the F queue with the specified number of players, each assigned unique identifiers. This setup eliminates manual player entry while ensuring diverse starting conditions.

Algorithm 5 CreateF

- 1: **Input:** F : Queue pointer, n : integer
 - 2: **Output:** None
 - 3: // Create n players with random names and ages
 - 4: // Add each player to queue F
 - 5: **Complexity:** $O(n)$ where n is number of players
-

2. **The function GameRound** manages a complete game session between two players, including turns, tracking and displaying scores. The function implements the game logic with maximum turn limits and a three-point gap win condition depending on the strategy (1 or 2).

Algorithm 6 GameRound

- 1: **Input:** g : GameState pointer, P1 : Player, P2 : Player
 - 2: **Output:** game : Game
 - 3: // Initialize game structure
 - 4: // Play up to maxturns (12 or 16)
 - 5: // Each turn calls PlayTurn or PlayTurn2
 - 6: // Check for 3-point gap condition
 - 7: // Display results
 - 8: **Complexity:** $O(t \cdot f)$ where t is number of turns (max 16) and f is complexity of PlayTurn/PlayTurn2
-

3. **The function MovePlayer** determines where a player should be placed after a game based on their status and the active strategy. The function implements the rules governing transitions between queues F, F1, F3 and lists LG, LP. It handles special cases like current winner management and definitive winner/loser qualification.

Algorithm 7 MovePlayer

- 1: **Input:** g : GameState pointer, P : Player pointer
 - 2: **Output:** None
 - 3: // Check player status (winner/loser)
 - 4: // Apply rules based on strategy (1 or 2)
 - 5: // Move player to appropriate queue/list
 - 6: // Handle current winner logic
 - 7: **Complexity:** $O(1)$ for queue operations, $O(n)$ for list insertions
-

4. **The function Priority** pairs players according to specific priority rules involving the current winner and three queues (F, F1, F3). The function systematically checks different combinations in predetermined order to select two opponents . It returns false only when no valid pairing can be made from available players.

Algorithm 8 Priority

```

1: Input: g : GameState pointer, P1 : Player pointer, P2 : Player pointer
2: Output: Boolean
3: // Check all possible player combinations according to priority rules
4: // 1. Current winner combinations
5: if g.currWinner  $\neq$  NULL then
6:   P1  $\leftarrow$  g.currWinner
7:   if NOT EmptyQueue(g.F1) then
8:     DeQueue(g.F1, P2)
9:     Print "Players chosen : Current Winner + F1 Player"
10:    return TRUE
11:  end if
12:  if NOT EmptyQueue(g.F) then
13:    DeQueue(g.F, P2)
14:    Print "Players chosen : Current Winner + F Player"
15:    return TRUE
16:  end if
17:  if NOT EmptyQueue(g.F3) then
18:    DeQueue(g.F3, P2)
19:    Print "Players chosen : Current Winner + F3 Player"
20:    return TRUE
21:  end if
22:  Print "Pairing with current winner impossible ."
23:  return FALSE
24: end if
25: // 2. F1 player combinations
26: // 3. F player combinations
27: // 4. F3 player combinations
28: // 5.No players available
29: Print "No players Available ."
30: return FALSE
31: // Each check involves queue operations
32: Complexity:  $O(1)$  (constant number of queue checks)

```

5. **The function Update_P_Stats** modifies player attributes after a game concludes, updating win/loss counts and streaks. Winners have their consecutive win streak incremented while losers have theirs reset to zero. It is crucial for determining player progression through queues.

Algorithm 9 Update_P_Stats

- 1: **Input:** winner : Player pointer, loser : Player pointer
 - 2: **Output:** None
 - 3: // Update winner statistics
 - 4: **Step 1: Update winner statistics**
 - 5: winner.nb_wins $\leftarrow$ winner.nb_wins + 1
 - 6: winner.nb_succ_wins $\leftarrow$ winner.nb_succ_wins + 1
 - 7: winner.nb_succ_losses $\leftarrow$ 0
 - 8: winner.succesifs_points $\leftarrow$ winner.succesifs_points + winner.game_score
 - 9: // Update loser statistics
 - 10: **Complexity:** $O(1)$ All operations are simple arithmetic assignments and field updates with constant time execution regardless of game size or player count. The function performs exactly 8 assignment operations and 4 arithmetic operations, making it highly efficient for frequent calls during tournament execution.
-

6. **The function GameResults** processes the outcome of a completed game round, determining winner and loser. The function updates scores, calls statistical update functions, and triggers player movement to appropriate queues. It also handles tie scenarios where neither player gains an advantage.

Algorithm 10 GameResults

- 1: **Input:** g: GameState pointer, game : Game, P1: Player pointer, P2: Player pointer
 - 2: **Output:** None
 - 3: // Determine winner/loser
 - 4: // Update scores
 - 5: // Update statistics
 - 6: // Move players accordingly
 - 7: // Handle tie case
 - 8: **Complexity:** $O(1)$ + complexity of called functions
-

7. **The function DisplayGameState** provides the current game situation including all data structures. The function shows queue and lists contents, current winner details, and overall game statistics; helping track game progress.

Algorithm 11 DisplayGameState

- 1: **Input:** g : GameState pointer
 - 2: **Output:** None
 - 3: // Display all queues
 - 4: // Display current winner
 - 5: // Display winners list (LG)
 - 6: // Display losers list (LP)
 - 7: **Complexity:** $O(n + m + p)$ where n, m, p are sizes of queues and lists
-

8. **The function FinishedGame** evaluates whether the tournament should conclude by checking if all players have reached definitive status. The function counts players in both winners and losers lists and compares with the total participant count. This termination condition ensures the game ends when no more matches can occur.

Algorithm 12 FinishedGame

- 1: **Input:** g : GameState pointer
 - 2: **Output:** Boolean
 - 3: // Count players in LG and LP
 - 4: // Compare with total players
 - 5: **Complexity:** $O(n + m)$ where n is players in LG, m is players in LP
-

9. **The function DisplayTop3Winners** highlights the three best performers from the winners list based on their consecutive points.

Algorithm 13 DisplayTop3Winners

- 1: **Input:** LG : List
 - 2: **Output:** None
 - 3: // Traverse LG list to display top 3 players
 - 4: **Complexity:** $O(1)$ (only checks first 3 elements)
-

10. **The function DISPLAY_N_WINS** allows to display at the end of the game, the players belonging to the LG and LP lists who have obtained n wins. It browses the two player lists and displays the information for each player. Similar to function Display_N_LOSSES.

Algorithm 14 DISPLAY_N_WINS

```

1: Input : LG, LP : List ; n : integer;
2: Output : display of players with exactly n wins
3: Var : found : integer ; current : pointer;
4: if (LG = NULL) AND (LP = NULL) then
5:   write ("There are no players on the LG and LP lists");
6: end if
7: // browsing LG list
8: current  $\leftarrow$  LG.L; // first element of the list LG
9: found  $\leftarrow$  0;
10: while current  $\neq$  NULL do
11:   if current.nb_wins > n then
12:     write ("No player in LG with n wins!");
13:   else
14:     if current.nb_wins = n then
15:       write ("Players with n win(s) in LG list:");
16:       found  $\leftarrow$  found + 1;
17:     end if
18:   end if
19:   current  $\leftarrow$  current.next;
20: end while
21: if found = 0 then
22:   write ("No players found in LG with n wins!");
23: end if
24: // browsing LP list
25: current  $\leftarrow$  LP.L; // first element of LP
26: found  $\leftarrow$  0;
27: while current  $\neq$  NULL do
28:   if current.nb_wins = n then
29:     ("Players who have n wins in LP are: ");
30:     found  $\leftarrow$  found + 1 ;
31:   end if
32:   current  $\leftarrow$  current.next;
33: end while
34: if found = 0 then
35:   write ("No players found in LP with n wins!");
36: end if

```

- The first while loop browses the LG list and may iterate through all its elements. Therefore, its complexity is linear $O(n_1)$.
- The second while loop does the same thing as the first, using the LP list. And therefore its complexity is $O(n_2)$.
- Finally, the overall complexity of the function is : $O(n_1 + n_2)$.

11. The function PlayGame

Algorithm 16 PlayGame - Complexity Analysis

```
1: Input:  $g$  : GameState pointer,  $\text{nbrPlayers}$  : integer
2: Output: None
3: // Initialize game
4: // Create initial queue
5: // Game loop for Part 1 (Strategy 1)
6: // Display intermediate results
7: // Game loop for Part 2 (Strategy 2)
8: // Clean up remaining players
9: // Display final results
10: Complexity:  $O(g \cdot (t + o))$  where  $g$  is number of games,  $t$  is turns per game,  $o$  is
    other operations
11: Primary Complexity Factors:
12: • Players:  $n$ , Games:  $g \leq 5n$ , Turns/game:  $t \leq 16$ 
13: • Queue/list ops:  $O(k)$  for current size  $k$ 
14:
15: Worst-Case Analysis:
16: Each game  $i$ :
17: 1. Priority():  $O(1)$ 
18: 2. GameRound():  $O(t \cdot f)$  where  $f$  is turn complexity
19: 3. MovePlayer():  $O(k)$  for list insertion
20: 4. DisplayGameState():  $O(n)$ 
21:
22: Since  $g_{\max} = 5n$  and per-game cost  $\leq O(n)$ :
23: Total:  $O(5n \cdot n) = O(n^2)$ 
24:
```

2.4 Conclusion

Studying the design and algorithms of our game allowed us to organize our project effectively, both in terms of code and algorithms. These elements will serve as the basis for the next chapter, where we will present the game's gameplay and results in the terminal.

Chapter 3

Implementation and realization

3.1 Introduction

In this chapter, we present the gameplay and mechanics of the game, as well as an analysis of the results obtained. We then discuss the main difficulties encountered during the project, and the proposed avenues for improvement.

3.2 Gameplay

3.2.1 Strategy One

Initially, all players are placed in the main queue F. Queues F1 and F3 are initially empty.

In each round, two players are selected in the following order: first from F1 (if it is not empty), otherwise from F, and finally from F3 if the other two are empty.

Once the players are selected, a random number is generated for each of them. If the sum of a player's digits is a multiple of 5, they score one point and immediately play another round, while the losing player is placed back at the end of the queue.

The game continues in this way until the difference between the two players reaches 3 points, or after twelve random values are generated if the difference is not reached. In case of a tie, both players are placed at the end of the queue.

When a player wins 3 rounds, they are placed in F1. If they lose 3 rounds, they are placed in F3. When a player wins 5 games in total, even if separated, he is placed in the LG ordered list, while a player who loses 5 games is placed in the LP list.

3.2.2 Strategy Two

From 3n games onwards, the second strategy begins. In each game, a random number is generated for both players. The greatest common divisor (GCD) is then calculated. If it contains at least one digit present in one of the two generated numbers, the player in question scores one point.

The game continues in this way until the difference between the two players reaches 3 points, at which point the winning player moves to queue F1 and the losing player to queue F3. Alternatively, after 16 random numbers have been generated, if the difference is not reached, the player with the highest score is placed in queue F, while the other player is placed in queue F3. In case of a tie, both players are placed at the end of the queue.

When a player wins a total of 2 games, they are placed in the ordered list LG. If they lose 2 games, they are placed in the list LP. The order of player selection for each round remains the same as in strategy 1: first in F1, then F, and finally F3.

3.3 Analysis of Results

Analysis of the game reveals a perplex overall fairness.

The game demonstrates commendable fairness by initially placing the players in identical conditions (all in F queue with zero stats and subject to same victory conditions(3-point difference or turn limits) and having the same probability of winning a round since it is based on random number generation. However, the notion of current winner and the priority queue system creates a hierarchy that rewards sustained performance rather than separated victories, further emphasized by the LG ranking (according to succesif_points).

In the first strategy, players are ranked in the LG and LP lists after winning or losing 5 games, not all players play the same number of games, and those who remain in first place benefit from priority selection. This heavily favors players who have won several consecutive games, although some players haven't had the same luck. Furthermore, this strategy is interrupted after 3*NumberOfPlayers, subjecting the remaining players to different rules.

In the second strategy, the game ends after 16 values are generated (instead of 12 in the first strategy), and the player with the highest score is placed directly in F1, while the loser goes to F3, bypassing the F list. Furthermore, a player is ranked in LG or LP as soon as they win or lose 2 games, whereas in the first strategy, 5 wins or losses were required. These rules accentuate the advantage of players who win consecutively and reduce the chances of others, making the game an unfair game of chance which cannot be influenced by skills or choices making the game seem like a lottery system.

3.3.1 Visual excerpts from the user experience

Here is the game as it appears in the terminal, showing the evolution of the players and their rankings in each game, as well as the progressive placement in the F1, F, F3, LG and LP queues and lists.

The program retrieves the number of players and the initial state of the game. Then, the first two players are selected from queue F. Next, the game begins with a series of successive rounds, during which the two strategies are applied until the end of the game (See figure below).

```

Hi, Dear Players !
Welcome to our game , conceived by Hamdi Sarah and Djenkal Lyna
To start playing may you please enter the total number of players!!!
5

Ok ! Now I invite you to enjoy :D

Start Game !!
Number of players : 5

Initial State of the game :

Current Game State :
Total games played : 0 | Strategy : 1
F Queue :
Player: Sarah
Player: Lyna
Player: Amine
Player: Imene
Player: Karim
F1 Queue :
The queue is empty
F3 Queue :
The queue is empty
The current Winner is :
No current winner .

The winners list :
Empty List.
The losers list :
Empty List.

Players chosen : F player + F Player
Start of Game 1 : Player 1 : Sarah VS Player 2 :Lyna

Beginning of round 1 :
Sarah's turn: You have generated the value : 211272 .The player 1 , Sarah scored a point !
Current: Sarah 1 - 0 Lyna
Lyna's turn: You have generated the value : 27544 .The player 2 , Lyna scored no point :(
Current: Sarah 1 - 0 Lyna

Beginning of round 2 :
Sarah's turn: You have generated the value : 850878 .The player 1 , Sarah scored no point :(
Current: Sarah 1 - 0 Lyna
Lyna's turn: You have generated the value : 777923 .The player 2 , Lyna scored a point !
Current: Sarah 1 - 1 Lyna

```

Figure 3.1: Game progress displayed from launch

The game ends after twelve consecutive values have been generated. The game results are displayed, followed by the final game status. Once this information is presented, the user presses Enter to play a new game with the current winner (See figure below).

```

Beginning of round 12 :
Sarah's turn: You have generated the value : 280278 .The player 1 , Sarah scored no point :(
Current: Sarah 4 - 2 Lyna
Game Results !!

THE WINNER IS : Sarah (P1)
Score: Sarah 4 - 2 Lyna

Game Results !
Player 1 , Sarah Wins : 4-2
Let's place the players !
Winner 1 , Sarah becomes the curr winner and will play next round .The player 2 , Lyna -> goes to F Queue .

Current Game State :
Total games played : 1 | Strategy : 1
F Queue :
Player: Amine
Player: Imene
Player: Karim
Player: Lyna
F1 Queue :
The queue is empty
F3 Queue :
The queue is empty
The current Winner is :
Player 1 , Sarah , Wins : 1 ,Consecutive wins : 1, Losses : 0 , Consecutive losses : 0 ,game score : 4 , score
4
The winners list :
Empty List.
The losers list :
Empty List.

Press 'ENTER' for next game :

End of Round
Part 1 games played : 1

Players chosen : Current Winner + F Player
Start of Game 2 : Player 1 : Sarah VS Player 3 :Amine

Beginning of round 1 :
Sarah's turn: You have generated the value : 351816 .The player 1 , Sarah scored no point :(
Current: Sarah 0 - 0 Amine

```

Figure 3.2: Display of rounds and winning player

After Strategy 1 is executed, the game results are displayed. The winning and losing players are indicated, along with the number of games won and lost (1, 2, or 3 games). The total number of games played and the number of players who participated are also displayed. Next, Strategy 2 begins, and the game state is displayed on the screen (See figure below).

```

End of Part I
Part I games played : 15
!!!!!!

Ended with participation of 5 players !

Let's display Part I results !

There is no player with 1 wins in the lists LG and LP
There is no player with 2 wins in the lists LG and LP
There is no player with 3 wins in the lists LG and LP

Losses Stats :

There is no player with 1 losses in the lists LG and LP
There is no player with 2 losses in the lists LG and LP
There is no player with 3 losses in the lists LG and LP

START OF PART 2 STRATEGY
-----
Press 'ENTER' to start Strategy 2 game :

Displaying PART II initial state :

Current Game State :
Total games played : 15 | Strategy : 2
F Queue :
Player: Lyna
Player: Imene
F1 Queue :
The queue is empty
F3 Queue :
Player: Sarah
Player: Karim
Player: Amine
The current Winner is :
No current winner .

The winners list :

```

Figure 3.3: End of the first strategy and display of results

The game is played using strategy 2. During execution, the time and date are displayed on the screen (See figure below).

```

Players chosen : F player + F Player
Start of Game 16 : Player 2 : Lyna VS Player 4 : Imene
Starting time is :on: 2026-01-17 at: 23h:48min:37sec

Beginning of round 1 :
Lyna's turn: You have generated the values: 48762 and 306633 .
The player 2 , Lyna scored a point !
Current: Lyna 1 - 0 Imene
Imene's turn: You have generated the values: 976197 and 576031 .
The player 4 , Imene scored a point !
Current: Lyna 1 - 1 Imene

Beginning of round 2 :
Lyna's turn: You have generated the values: 238588 and 860589 .
The player 2 , Lyna scored no point :(
Current: Lyna 1 - 1 Imene
Imene's turn: You have generated the values: 54873 and 998877 .
The player 4 , Imene scored a point !
Current: Lyna 1 - 2 Imene

Beginning of round 3 :
Lyna's turn: You have generated the values: 637304 and 979358 .
The player 2 , Lyna scored no point :(
Current: Lyna 1 - 2 Imene
Imene's turn: You have generated the values: 861200 and 777254 .
The player 4 , Imene scored a point !
Current: Lyna 1 - 3 Imene

Beginning of round 4 :
Lyna's turn: You have generated the values: 68960 and 14915 .
The player 2 , Lyna scored a point !
Current: Lyna 2 - 3 Imene
Imene's turn: You have generated the values: 293947 and 645480 .
The player 4 , Imene scored no point :(
Current: Lyna 2 - 3 Imene

Beginning of round 5 :
Lyna's turn: You have generated the values: 207730 and 669955 .
The player 2 , Lyna scored a point !
Current: Lyna 3 - 3 Imene
Imene's turn: You have generated the values: 674546 and 837107 .
The player 4 , Imene scored a point !
Current: Lyna 3 - 4 Imene

```

Figure 3.4: Gameplay using second strategy

Strategy 2 has ended; no more players are available. The last player in queue F3 is then moved to the LP list. At this point, the number of games played with Strategy 2, as well as the total number of games played, are displayed to indicate the progress and overall results of the game.

```

Current Game State :
Total games played : 20 | Strategy : 2
F Queue :
The queue is empty
F1 Queue :
The queue is empty
F3 Queue :
Player: Amine
The current Winner is :
No current winner .

The winners list :
1. Player 4, Imene | Age: 23 | Score: 21 | Wins: 2 | Losses: 2
2. Player 5, Karim | Age: 96 | Score: 28 | Wins: 2 | Losses: 3
The losers list :
1. Player 5, Karim | Age: 96 | Score: 37 | Wins: 2 | Losses: 4
3. Player 1, Sarah | Age: 59 | Score: 27 | Wins: 2 | Losses: 4
5. Player 4, Imene | Age: 23 | Score: 27 | Wins: 2 | Losses: 3
7. Player 2, Lyna | Age: 39 | Score: 17 | Wins: 1 | Losses: 3

Press 'ENTER' for next game :

End of Round
Part 2 games played : 5

No players Available .
Last Player in F3 : 3. Amine -> moves to LP

End of Part II
Part 2 games played : 5

End of Game
Total game rounds played : 20

END OF GAME RESULTS :
The top 3 winners of the game are :
-----
*drumroll Please*
1.Player 4, Imene | Age: 23 | Score: 21 | Wins: 2 | Losses: 2

```

Figure 3.5: End of the second strategy

At the end of the game, information including the number of wins and losses for each player is displayed. Before that, the top three players with the most wins are shown, allowing for a quick overview of the game's winners.

```

Win Stats :
Players with 0 win(s) in the list LG:
No player found in LG with 0 number of wins !
Players with 0 win(s) in the list LP:
No player found in LP with 0 number of wins !
Players with 1 win(s) in the list LG:
No player found in LG with 1 number of wins !
Players with 1 win(s) in the list LP:
In LP : Player: Lyna, Wins: 1
Players with 2 win(s) in the list LG:
In LG : Player: Imene, Wins: 2
In LG : Player: Karim, Wins: 2
Players with 2 win(s) in the list LP:
In LP : Player: Amine, Wins: 2
In LP : Player: Karim, Wins: 2
In LP : Player: Sarah, Wins: 2
In LP : Player: Imene, Wins: 2

Losses Stats :
Players with 1 loss(es) in the list LP:
No player found in LP with 1 number of wins !
Players with 1 loss(es) in the list LG:
No player found in LG with 1 number of wins !
Players with 2 loss(es) in the list LP:
No player found in LP with 2 number of wins !
Players with 2 loss(es) in the list LG:
In LG : Player: Imene, Losses: 2
No player found in LG with 2 number of wins !

-----
Thank you for playing our game ! :)
Program ended with exit code: 0

```

Figure 3.6: End of game results

3.4 Difficulties encountered and prospects for improvement

During the development of our project, we encountered several difficulties. First, decomposing the program into subroutines required careful consideration to clearly define the functions to be used and ensure proper coordination between them.

In terms of future improvements, we could integrate a database to save player information, instead of relying solely on file and list structures.

3.5 Conclusion

This chapter describes the game's development and illustrates its functionality during a gameplay session. The results obtained show that the game works as intended : A fair competitive system but requiring better rules to flesh out a truly equitable competition. Difficulties encountered during development have been identified, and potential avenues for future improvement have been briefly discussed.

Conclusion

The report presented the entire project, from the initial idea to the complete execution of the games. It details the implementation of the two strategies, the management of players in the various queues and lists, as well as the final rankings in LG and LP. The analysis of the results shows the evolution of the players throughout the games and the impact of the rules on the rankings and fairness. The displays in the terminal allow players to visualize the game dynamics and the score progression for each player, thus providing a complete representation of how the game works.